

Eksamen-IKT2004-02.06.26

Case

Et selskap som driver med brylupsplanlegging, vil lage en webbasert løsning.

selskapets ansatte skal kunne legge inn ulike produkter i ett admin-grensesnitt. Eksempler på produkter er musikere, bordkort, blomster etc. Hvert produkt skal ha navn, beskrivelse, pris, bilde. Disse produktene skal lagres i en database.

i tillegg skal det være en frontend for kunder, der de kan se listen over produkter, og for hvert produkt klikke inn på en detaljvisning.

Selskapet ønsker å drive løsningen på sine egne servere. Admin-grensesnittet skal kun være tilgjengelig fra datamaskiner som står på bedriftens eget nettverk.

min oppgave

Svaret jeg har gitt på oppgaven er en forenklet løsning som fungerer med det jeg kan og har kompetanse til. Jeg har brukt KI til mye av kodingen men jeg viser forståelse for bruk av KI, IDE, og hvordan man setter sammen koden. Oppgaven er en veldig utviklingsbasert oppgave, og dette er ikke min spesialitet, i løpet av oppgaven vil jeg prøve å lære og å få så stor forståelse som mulig. Jeg føler jeg har kontroll på hvordan webservere, docker, og markdown filer virker, og jeg føler jeg har god kontroll på brukerstøtte delene.

funksjonalitet

Nettsiden fungerer og følger oppgaven. Den har en side for kundene hvor automatisk alt fra databasen blir vist. Nettsiden har en admin side hvor du lett ved bruk av GUI-en kan legge til og fjerne rader i tabellen. Nettsiden oppfyller kravene til bedriften og har funksjon.

Filer laget med KI:

Her kommer en liste med filer som er laget av KI. Disse filene er de jeg føler jeg har minst kontroll på og de må jeg øve på. Full KI logg kan du finne i ai-logg.md, der vises det tydelig at jeg prøver å fi ki-en til å forklare istedenfor å gi fulle filer. Jeg føler også jeg viser forståelse for bruk av IDE, og hvordan alt knyttes opp til min database, da jeg ikke har kitt KI noe informasjon om min database.

- liste.ejs
- skjema.ejs
- detalj.ejs
- liste.ejs
- db.js
- server.js

Beskrivelse av andre filer

ai-logg.md

En logg av alle KI prompter og svar.

Brukerveiledning.md

En brukerveiledning til kunder om hvordan man vil kunne registrere fremtidige brukere, og å spore bestillinger.

Arkitektur.md

Hvilke pakker, apper, og containere som er brukt for opsett og drifting. Du kan også finne ett databse diagram her.

Drift.md

Hvordan IT avdelingen kan sette opp nettsiden selv og drifte den.

Sikkerhet.md

Sikkerhets problemer med nettsiden og hva som kan tenkes på.

README.md

Filen du leser i nå! her er alt du trenger å vite om besvarelsen og hvordan dette skal brukes.

Denne filen vil bli lengre og lengre utover eksamensdagen.

Arkitektur

Database diagram



Database

Databasen er satt opp som en docker container, denne containeren fungerer fint fordi den kan lettes sendes rundt og vil virke på alle maskiner. Det er en Postgres container på port 5432. Denne bør settes opp på en server som er 24/7 og passordet burde endres og fjernes fra kildekoden. Databasen har en tabell, denne tabellen, er produkter, på bildet over denne teksten kan du se hva som er i den tabellen. Jeg ser ikke noe behov for flere tabeller, mne om det skal bli kunde registrering senere må det bygges opp flere tabeller. Det kan også lages en bestillinger tabell om ansatte skal kunne logge segg inn og se bestillinger.

NPM pakker

NPM pakker er pakker med kode som gjør ting. dette kan lett implementeres i din egen kode, og gjør prosessen mye enklere.

Det er brukt pakkene express, pg, og ejs. Express brukes til å lage rutene og å gåndtere url-ene. pg brukes så node kan kobles opp til postgres og ejs brukes så det kan puttes informasjon fra databasen inn i nettsiden. *Hvilke pakker som skal lastes ned har blitt bestemt med hjelp av KI*

Det er blitt lagt til en gitignore fil så npm ikke følger med til github, dette kan lastes ned igjen med `npm install express pg ejs`

Web-server

Web-serveren som blir brukt er bare å serve filen med node, ikke bra nok for en ekte bedriftsløsning, men fungerer til å teste og jobbe med det. I den virkelige verden ville jeg brukt en server som kjører, f. eks NGINX.

Drift

Dette er en fil for hvordan IT skal kunne deploy nettsiden og faktisk administrere den.

opsett

For å sette opp nettsiden må npm installeres med de riktige packages, disse finner du i arkitektur.md. Deretter må det startes en postgres docker container og passord og brukernavn og andre detaljer må endres i db.js.

For å kjøre web serveren må du peke node på server.js, da skriver du i en terminal som er åpen i "src" mappen `Node server.js`

Kontinuerlig driftig

Videre for at den skal kjøre skal det ikke være noe mer som trenger å gjøres, men jeg kan anbefale å bytte vekk fra node til NginX som kjører på en server. Denne serveren bør ekskluderes med DMZ til en egen del av nettverket så ikke folk akn hacke seg videre inn.

Den nåværende nettsiden har ett ganske stygt og dårlig design, dette kan fikses med css, da lager du en fil under public mappen som heter `style.css`

Sikkerhet

Det er mange sikkerhets feil i denne kildekoden, det har rett og slett noe å si med at jeg ikke vet nok, og er ikke profesjonell.

Ett av de tydeligste problemene er at databseserveren sitt passord og brukernavn står rett i kildekoden, noe som gjør dne veldig lett å nå og bruke. Dette kan føre til SQL injections. Admin siden er ikke veldig godt seperert fra nettet og kan lett brytes seg inn på.

fiks

For å fikse mange av disse kan man f. eks sette opp sterkere brannmurer. Det kan også hjelpe å bruke for eksempel en .env fil med variabler så den kan ligge i gitignore så passord og sånn til database ikke blir puttet rett ut på nettet.

Admin side

Akuratt nå så er ikke admin siden godt nok "skjult". For å gjøre at den kun er tilgjengelig via de ansatte sitt nett ville jag ha satt opp vlan i de ansatte sin bedrift og lyst ut admin siden på en egen port. Da lages det en portåpning på kun det vlan-et som skal ha tilgang, så man må være tilkoblet dette nettet for tilgang. Det er også mulig å sette opp en VPN for de ansatte som de kan koble seg på, så de kan nå siden fra hvor som helst.

Bruerveiledning

Dette er en del av oppgave 3 hvor jeg skal forklare hvordan registrering vil foregå om det eventuelt blir aktuelt i fremtiden.

Denne veiledningen er rettet mot kunder og brukere va nettsiden.

Veiledningen starter her

Dette er en veiledning for å sette opp en konto og å handle hos brylupsplanlegging AS. For å registrere en bruker navigerer du musepekeren til bruker ikonet i toppen av skjermen, når du trykker på dette ikonet vil du få frem ett registrering skjema. I dette skjemaet vil du fylle ut informasjon om deg, her vil du bli spurt om navn, epost, telefonnummer, og ett passord. Dette passordet er det viktig du ikke deler med folk, og det må huskes så du kan komme deg inn senere.

Etter alt er fylt ut trykker du på "registrer bruker". Etter dette vil du komme tilbake til hjemmesiden, der kan du handle som normalt. Etter du har funnet produktene du vil ha og lagt dem til i handlekurven vil du kunne se i utsjekke at informasjonen din allerede er fylt ut, om den ikke er det så er du blitt logget ut, da går du tilbake til det første steget og fullfører registreringen igjen. Etter du har lagt inn bestillingen din kan du igjen trykke på bruker ikonet i toppen av skjermen din. Her inne vil du kunne se "mine bestillinger". I denne seksjonen kan du spore bestillingene dine og du har en oversikt over hva som har blitt bestilt tidligere.

Dersom du senere skulle trenge å logge deg inn på denne siden for å se bestillingene dine trykker du på bruker ikonet i toppen igjen. Om du er blitt logget ut Fyller du inn informasjonen og trykker "logg inn". Har du glemt passordet ditt trykker du "glemt passord?" og følger prosessen. Når du er inne vil du kunne se bestillingene dine igjen

GDPR

Denne filen er en del av brukerstøtte oppgaven, her kan du lese om GDPR og håndtering av brukere på sikt.

Dersom det skal settes opp ett konto system på nettsiden i fremtiden må dataen behandles riktig, dataen skal helst behandles kryptert uten mulighet for å kunne se på det. Når det kommer til bestillinger Må ting som adresse, navn, osv behandles på en forsvarlig måte. dette innebærer å f. eks ikke dele til flere enn nødvendig, lage en form for taushetsplikt i bedriften for alle ansatte, og å agere dataen på en sikker måte og ikke på hvem som helst sin pc.

Dersom bedriften ikke følger GDPR reglene kan det i værste fall føre til straff, og vil føre til dårlig rykte, det kan derfor også være greit med interne kurs om hvordan data skal brukes og behandles. En annen ide kan være phishing simulering for å sjekke at folk faktisk følger reglene og tingene de blir lært på kurset.

Lagret info

Run Postgres

Hentet fra:

<https://tangen-2it-utvikling.netlify.app/content/database-01#/4> |

01. 06. 2026

```
docker run --rm -p5432:5432 -e POSTGRES_PASSWORD=mysecretpassword postgres
```

Postgres tilkobling

Hentet fra:

<https://tangen-2it-utvikling.netlify.app/content/database-01#/8> |

01. 06. 2026

server: localhost

brukeravn: postgres

port: 5432

passord: mysecretpassword

database: postgres

Mal for å lage tabell

Hentet fra:

<https://tangen-2it-utvikling.netlify.app/content/database-01#/19> |

01. 06. 2026

```
CREATE TABLE elev (  
  id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  navn VARCHAR(100) NOT NULL,  
  alder INTEGER  
);
```

Mal for å legge til i tabell

<https://tangen-2it-utvikling.netlify.app/content/database-01#/20> |

01. 06. 2026

```
INSERT INTO elev (navn, alder) VALUES ('Ola Tveiten', 19);
```

Handelnettsider for utstyr

Komplett

<https://www.komplettbedrift.no/>

Dustin

<https://www.dustin.no/>

Atea eShop

<https://www.atea.no/eshop/>

KI-Logg

Chat 1 (Claude opus 4.8)

Prompt

Jeg har fått en oppgave og jeg er veldig lost på utvikling delen, jeg vil at du skal hjelpe meg med den, også forklare så jeg får en stor grad forståelse og ikke bare at du gjør det.

"Et selskap som driver med brylupsplanlegging, vil lage en webbasert løsning.

selskapets ansatte skal kunne legge inn ulike produkter i ett admin-grensesnitt. Eksempler på produkter er musikere, bordkort, blomster etc. Hvert produkt skal ha navn, beskrivelse, pris, bilde. Disse produktene skal lagres i en database.

i tillegg skal det være en frontend for kunder, der de kan se listen over produkter, og for hvert produkt klikke inn på en detaljvisning.

Selskapet ønsker åm drifte løsningen på sine egne servere. Admin-grensesnittet skal kun være tilgjengelig fra dataskinner som står på bedriftens eget nettverk."

"Oppgave 1 Utvikling

- database opsett - struktur og SQL for å opprette tabeller
- Kundens web løsning - en enkel visning av listen av produkter samt detaljvisning, med HTML og CSS
- admin-grensesnitt - CRUD funksjonalitet for å opprette og redigere produkter
- Dere kan velge emllom Node.js/Express, eller andre rammeverk

"

Så jeg vet ikke noe om den delen, jeg kan sikkert fikse drift og brukerstøtte, men jeg trenger hjelp med hvor jeg skal starte. du kan se hva jeg har klart her:

<https://github.com/08Robfin/Eksamen-IKT2004-02.06.26/blob/main/docs/bilder/ProdukterTabell.png>

jeg har satt opp en postgres docker container. med denne tabellen

Velagt bilde av tabelldiagram

Svar

Yo Robin 🙌 La oss rydde opp i kaoset. Du er faktisk lenger enn du tror — du har **databasen oppe** allerede, som er steg 1. Resten er bare å bygge lag på lag oppå den.

La meg gi deg **kartet** først, så starter vi der det gir mening.

Hvordan henger alt sammen?

Tenk på løsningen som 3 lag:

Postgres (data) → Express-server (logikk) → HTML/CSS (det brukeren ser)

Siden oppgaven krever **admin-innlogging, databasekall og nettverkssperre** (alt sammen ting som MÅ skje på serveren, ikke i nettleseren), er **Node.js + Express med server-side rendering** den perfekte matchen. Du har gjort akkurat dette på skolen før, så dette blir hjemmebane 😎

Konkret bruker du:

- **pg** — biblioteket som lar Node snakke med Postgres
- **EJS** — en templating-motor som lar deg sprøyte data inn i HTML (`<%= produkt.navn %>`)
- **Express middleware** — for nettverkssperra på admin

Steg 1: Fiks tabellen din 🛠️

Tabellen din er **90% riktig**, men la meg roast deg litt så du lærer 😊

Du skrev `Pris -- Decimal` ✅ — **bra jobba**, du brukte IKKE `float`. Float på penger = avrundingshelvete ($0.1 + 0.2 = 0.30000004$). Du slapp unna roast der.

Men her er forbedringene:

```
CREATE TABLE produkter (  
  id          INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  produktnavn VARCHAR(255) NOT NULL,  
  pris        NUMERIC(10,2) NOT NULL,  
  beskrivelse TEXT,  
  bilde       VARCHAR(255)  
);
```

Hva endret jeg og hvorfor:

- **GENERATED ALWAYS AS IDENTITY** istedenfor **SERIAL**. Begge gir auto-inkrement, men **IDENTITY** er den moderne SQL-standard-måten. **SERIAL** fungerer fortsatt, men er litt "gammeldags" i nyere Postgres.
- **NUMERIC(10,2)** = maks 10 siffer totalt, 2 etter komma. Altså opptil 99 999 999.99 kr. Du MÅ gi presisjon, ellers vet ikke databasen hvor mange desimaler den skal lagre.
- **beskrivelse TEXT** istedenfor varchar — beskrivelser kan bli lange, og **TEXT** har ingen lengdegrense. (I Postgres er **VARCHAR** uten tall egentlig = **TEXT**, men vær eksplisitt.)
- **bilde VARCHAR(255)** — du lagrer **filstien/filnavnet** til bildet her (f.eks. `roser.jpg`), IKKE selve bildet. Selve bildefila ligger i en mappe på serveren. Det var riktig tenkt av deg 👍
- **NOT NULL** på navn og pris — et produkt uten navn eller pris gir ingen mening, så vi tvinger dem til å være der.

Steg 2-4: Resten av byggingen

Etter databasen bygger vi i denne rekkefølgen:

1. **Sett opp Express-serveren** + koble til Postgres med `pg`
2. **Kundesiden** — `GET /` (liste) og `GET /produkt/:id` (detaljvisning)
3. **Admin-CRUD** — opprette, lese, endre, slette produkter
4. **Nettverkssperra** — en middleware som sjekker om IP-en til den som kobler seg på er på bedriftens lokalnett (f.eks. `192.168.x.x`), og blokkerer admin hvis ikke. Dette er ren **nettverk-logikk**, så dette kommer du til å digge 🌐

Vil du at vi starter med **Steg 2** (Express + DB-tilkobling) nå? Da bygger vi opp prosjektet fra bunn, og jeg forklarer hver bit mens vi går — ikke bare dumper kode på deg. 🚀

Prompt

Jeg vet ikke hvordan jeg setter opp noe av det du sier, eller hvordan jeg lager en nettside, den skal være simpel, men har ingen ide om hvordan den tar ting fra databasen inn på nettsiden, og vet heller ikke hvordan hvert produkt kan få sin egen side?

Svar

Helt good, dette er akkurat det folk sliter med i starten 🙌 La meg knekke **mentale modellen** for deg først, for når den klikker, blir resten lett. La meg tegne det opp. Helt logisk at det føles magisk — "hvordan vet nettsiden hva som ligger i databasen?" La meg vise deg hva som faktisk skjer hver gang noen åpner en produktside 📌 Den greia i steg 6 er hele poenget: serveren bygger **ferdig HTML** før den sender den til nettleseren. Nettleseren får aldri se databasen — den får bare en helt vanlig HTML-side som serveren har "fylt ut" med data. Det kalles **server-side rendering (SSR)**. 🧠

La oss faktisk bygge det. Følg med, jeg forklarer hver bit.

1. Sett opp prosjektet

I terminalen, i en ny mappe:

```
npm init -y
npm install express pg ejs
```

- **express** = serveren (håndterer at folk besøker URL-er)
- **pg** = lar Node snakke med Postgres
- **ejs** = lar deg sprøyte data inn i HTML

Mappestruktur du skal lage:

```
prosjekt/  
├── db.js           ← kobling til databasen  
├── server.js       ← selve serveren + ruter  
├── views/  
│   ├── liste.ejs  
│   └── detalj.ejs  
└── public/  
    └── style.css   ← CSS og bilder
```

2. Koble til databasen — `db.js`

```
const { Pool } = require('pg');  
  
const pool = new Pool({  
  host: 'localhost',  
  port: 5432,  
  user: 'postgres',  
  password: 'passordet_ditt',  
  database: 'navnet_paa_databasen'  
});  
  
module.exports = pool;
```

Dette er bare "telefonnummeret" til databasen. `pool` er en gjenbrukbar tilkobling vi kan importere overalt. Bytt ut passord og databasenavn med det du satte i Docker-containeren din.

3. Serveren + de to rutene — `server.js`

Her skjer magien. Les kommentarene 🙌

```

const express = require('express');
const pool = require('./db');
const app = express();

app.set('view engine', 'ejs'); // si til Express: bruk EJS for HTML
app.use(express.static('public')); // server CSS/bilder fra /public

// KUNDESIDEN: liste over ALLE produkter
app.get('/', async (req, res) => {
  const resultat = await pool.query('SELECT * FROM produkter ORDER BY id');
  res.render('liste', { produkter: resultat.rows });
});

// DETALJVISNING: ETT produkt, basert på id i URL-en
app.get('/produkt/:id', async (req, res) => {
  const id = req.params.id;
  const resultat = await pool.query(
    'SELECT * FROM produkter WHERE id = $1', [id]
  );

  if (resultat.rows.length === 0) {
    return res.status(404).send('Fant ikke produktet');
  }

  res.render('detalj', { produkt: resultat.rows[0] });
});

app.listen(3000, () => console.log('Kjører på http://localhost:3000'));

```

Det viktigste her:

- `app.get('/', ...)` = "når noen besøker forsida, kjør denne koden"
- `pool.query(...)` = send SQL til databasen, få rader tilbake (`resultat.rows`)
- `res.render('liste', { produkter })` = ta malen `liste.ejs`, fyll den med dataen, send ferdig HTML

4. Hvordan får hvert produkt sin egen side? 🤖

Dette er spørsmålet ditt, og svaret er **én linje**: `/produkt/:id`

`:id` er en **placeholder** (jokertegn). Du lager IKKE én side per produkt manuelt. Du lager **én route** som funker for alle:

- Noen går til `/produkt/3` → `req.params.id` blir `3` → henter produkt 3
- Noen går til `/produkt/47` → `req.params.id` blir `47` → henter produkt 47

Samme kode, samme mal — bare ulik data avhengig av tallet i URL-en. Det er derfor du kan ha 10 000 produkter med én eneste fil. 🔥

Roast-tid 😏: Se at jeg skrev `WHERE id = $1` med `[id]` ved siden av? Det MÅ du gjøre. Ikke frist med å lime id rett inn i strengen sånn:

```
// ALDRI gjør dette ❌  
pool.query(`SELECT * FROM produkter WHERE id = ${id}`);
```

Da har du laget en åpen dør for **SQL-injection** — noen kan skrive `1; DROP TABLE produkter` i URL-en og slette hele tabellen din. `$1` lar Postgres behandle input som ren data, ikke som kommandoer. Med tanke på wardriving-prosjektet ditt vet du jo hvorfor sånt er kjipt. 🛡️

5. Malene — `views/liste.ejs`

```

<!DOCTYPE html>
<html lang="no">
<head>
  <meta charset="UTF-8">
  <title>Våre produkter</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <h1>Våre produkter</h1>
  <div class="produktliste">
    <% produkt.forEach(produkt => { %>
      <a class="kort" href="/produkt/<%= produkt.id %>">
        <h2><%= produkt.produktnavn %></h2>
        <p><%= produkt.pris %> kr</p>
      </a>
    <% }) %>
  </div>
</body>
</html>

```

To EJS-symboler å lære:

- `<%= ... %>` = skriv ut en verdi (f.eks. produktnavnet)
- `<% ... %>` = kjør JavaScript-logikk (f.eks. en `forEach`-løkke)

Løkka her lager ett `<a>`-kort per produkt automatisk. Og legg merke til `href="/produkt/<%= produkt.id %>"` — det er DETTE som lager lenka til detaljsida for akkurat det produktet. 🎯

Og `views/detalj.ejs`:


```
<!DOCTYPE html>
<html lang="no">
<head>
  <meta charset="UTF-8">
  <title><%= produkt.produktnavn %></title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <a href="/">← Tilbake</a>
  <h1><%= produkt.produktnavn %></h1>
  <p class="pris"><%= produkt.pris %> kr</p>
  <p><%= produkt.beskrivelse %></p>
</body>
</html>
```

6. Kjør det

```
node server.js
```

Åpne `http://localhost:3000` i nettleseren. Hvis du har lagt inn noen testrader i tabellen, ser du dem nå — og du kan klikke deg inn på hvert produkt. 🎉

(Har du ikke testdata enda? Kjør en rask `INSERT INTO produkter (produktnavn, pris, beskrivelse) VALUES ('Roser', 499.00, 'Vakker bukett');` i databasen først.)

Når dette kjører og du forstår flyten, er neste steg **admin-grensesnittet** (CRUD — altså å legge til/endre/slette produkter via et skjema). Det bruker samme prinsipp, bare med `POST`-ruter i tillegg.

Vil du at vi tar admin-delen nå, eller vil du få denne kundesida til å kjøre først og leke deg litt med den? 🚀

Prompt

why does this not show an image?

-- første tabell hentet og redigert fra: <https://tangen-2it-utvikling.netlify.app/content/database-01#>

| 01. 06. 26

```
CREATE TABLE Produkter (  
  id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  Produktnavn VARCHAR(100) NOT NULL,  
  Pris DECIMAL(10, 2) NOT NULL,  
  Beskrivelse VARCHAR(255),  
  Bilde VARCHAR(255)  
);  
  
INSERT INTO produkter (Produktnavn, Pris, Beskrivelse, Bilde) VALUES ('Roser', 29.99, 'En  
bukett med vakre roser, Pakken inneholder 3 roser.', './public/bilder/roser.webp');
```

Svar

Fordi du har lagret en **diskbane** i databasen, men nettleseren trenger en **URL**. Det er to helt forskjellige ting. 🧠

Hvorfor det knekker

Når du skrev `app.use(express.static('public'))`, sa du til Express: *"alt inni public-mappa skal serveres fra web-rota /"*. Express **fjerner** `public`-biten automatisk. Så:

På disken (filsystemet)	URL i nettleseren
<code>public/bilder/roser.webp</code>	<code>/bilder/roser.webp</code>

Du lagret `./public/bilder/roser.webp` i databasen. Når den havner i ``, prøver nettleseren å finne fila relativt til sida du står

på. Står du på `/produkt/3`, leter den etter `/produkt/public/bilder/roser.webp` — som ikke finnes. Død lenke. 💀

Roast 😏: databasen skal ikke vite noe om mappestrukturen på serveren din. Å blande diskbaner (`./public/...`) og web-URL-er er den klassiske nybegynnerfella. Hold dem adskilt.

Fiksen

Lagre **bare filnavnet** i databasen, og bygg URL-en i malen. Da er databasen "ren" og du kan flytte bildemappa uten å endre dataene.

Oppdater dataene dine:

```
UPDATE produkter SET Bilde = 'roser.webp' WHERE Produktnavn = 'Roser';
```

Og for nye rader fremover:

```
INSERT INTO produkter (Produktnavn, Pris, Beskrivelse, Bilde)
VALUES ('Roser', 29.99, 'En bukkett med vakre roser. Pakken inneholder 3
roser.', 'roser.webp');
```

I malen (`liste.ejs` / `detalj.ejs`) setter du `/bilder/` foran selv:

```
">
```

Sjekkliste før du gir opp 🔍

1. **Ligger fila faktisk på disken?** Den må være her: `public/bilder/roser.webp`. Har du laget `bilder`-mappa inni `public`? Ligger `roser.webp` der?

2. **Riktig navn og endelse?** `roser.webp` $\neq$ `Roser.webp` $\neq$ `roser.png`. Linux/Docker bryr seg om store/små bokstaver.
3. **Test URL-en direkte:** åpne `http://localhost:3000/bilder/roser.webp` i nettleseren. Ser du bildet der, fungerer static-serveringen — da er feilen i `<img>`-taggen. Får du 404, ligger ikke fila riktig på disken. 🎯

Test punkt 3 først — den forteller deg på 2 sekunder om problemet er **fila** eller **koden**. 💪

Prompt

url-en virker

<http://localhost:3000/bilder/roser.webp>

Svar

Da vet vi nøyaktig hva som er galt: **static-serveringen fungerer perfekt, fila ligger riktig**. Feilen er i `<img>`-taggen — altså hvilken URL EJS faktisk lager. 🎯

Sannsynligvis skjer én av disse:

Databasen din har fortsatt den gamle verdien `./public/bilder/roser.webp`. Så hvis malen din er ``, blir resultatet:

```
 <!-- søppel ✗ -->
```

Gjør dette nå

1. **Fiks dataene** (kjør i databasen):

```
UPDATE produkter SET Bilde = 'roser.webp' WHERE Produktnavn = 'Roser';
```

2. Sjekk at malen er sånn:

```
">
```

3. Last inn sida på nytt, og bruk superkraften din: **høyreklikk** → **"Vis kildekode"** (eller F12 → Inspiser). Finn `<img>`-taggen og se hva `src` faktisk ble. 🔍

Du vil at den skal stå nøyaktig sånn:

```

```

Akkurat den URL-en som funket da du testa direkte. 💪

Det å lese **generert HTML** i nettleseren er den viktigste debugging-vanen du kan lære deg i webutvikling — for den viser deg hva serveren *faktisk* sendte, ikke hva du *trodde* du sendte. Forskjellen mellom de to er der 90% av bugsa lever. 🐛

Si ifra hva `src` faktisk ble, så fikser vi det med en gang hvis det fortsatt er rart.

Prompt

ok, nå mangler vi en funksjonell admin side, som kun skal være tilgjengelig lokalt. hvordan fikser vi dette?

Svar

To jobber her: først gjøre admin-sida **funksjonell** (CRUD), så **låse døra** så bare bedriftens nett slipper inn. La oss ta dem i rekkefølge. 🔒

Den nye ideen: skjemaer sender data TILBAKE

Hittil har serveren bare *sendt ut* HTML (`GET`). Nå skal nettleseren *sende inn* data — når noen fyller ut et skjema og trykker "Lagre". Det gjør den med `POST`.

Tommelfingerregel: `GET` = **hente/vis** (les), `POST` = **endre noe** (skriv).

For at Express skal kunne lese skjemadata, legg til denne linja øverst i `server.js` (rett under `app.use(express.static(...))`):

```
app.use(express.urlencoded({ extended: true })); // les skjemadata inn i req.body
```

Glemmer du denne, blir `req.body` `undefined` — klassisk bug som tar folk en time å finne. 🐛

CRUD-rutene — i `server.js`

CRUD = Create, Read, Update, Delete. Her er alle:

```

// ===== ADMIN =====

// READ: liste med rediger/slett-knapper
app.get('/admin', async (req, res) => {
  const r = await pool.query('SELECT * FROM produkter ORDER BY id');
  res.render('admin/liste', { produkter: r.rows });
});

// Vis tomt skjema for nytt produkt
app.get('/admin/ny', (req, res) => {
  res.render('admin/skjema', { produkt: null });
});

// CREATE: lagre nytt produkt
app.post('/admin/ny', async (req, res) => {
  const { produktnavn, pris, beskrivelse, bilde } = req.body;
  await pool.query(
    'INSERT INTO produkter (produktnavn, pris, beskrivelse, bilde) VALUES'
    ($1, $2, $3, $4)',
    [produktnavn, pris, beskrivelse, bilde]
  );
  res.redirect('/admin');
});

// Vis forhåndsfyllt skjema for redigering
app.get('/admin/:id/rediger', async (req, res) => {
  const r = await pool.query('SELECT * FROM produkter WHERE id = $1',
    [req.params.id]);
  res.render('admin/skjema', { produkt: r.rows[0] });
});

// UPDATE: lagre endringer
app.post('/admin/:id/rediger', async (req, res) => {
  const { produktnavn, pris, beskrivelse, bilde } = req.body;
  await pool.query(
    'UPDATE produkter SET produktnavn=$1, pris=$2, beskrivelse=$3,
    bilde=$4 WHERE id=$5',
    [produktnavn, pris, beskrivelse, bilde, req.params.id]
  );
  res.redirect('/admin');
});

// DELETE: slett produkt
app.post('/admin/:id/slett', async (req, res) => {
  await pool.query('DELETE FROM produkter WHERE id = $1',
    [req.params.id]);
  res.redirect('/admin');
});

```

Tre ting verdt å skjønne:

- `const { produktnavn, pris } = req.body` — `name`-attributtet på hvert input-felt blir nøkkelen her. Et `<input name="pris">` havner i `req.body.pris`.
- `$1, $2, ...` — samme parameterisering som før. Aldri lim brukerinput rett inn i SQL.

- `res.redirect('/admin')` — etter en `POST` sender vi brukeren tilbake til lista. Dette kalles **Post/Redirect/Get** og hindrer at en sideoppdatering sender skjemaet på nytt (så du ikke får 5 roser igjen 😊).

Malene

Lag mappa `views/admin/`. Først `views/admin/liste.ejs`:


```

<!DOCTYPE html>
<html lang="no">
<head><meta charset="UTF-8"><title>Admin</title><link rel="stylesheet"
href="/style.css"></head>
<body>
  <h1>Admin: produkter</h1>
  <a href="/admin/ny">+ Nytt produkt</a>
  <table>
    <tr><th>Navn</th><th>Pris</th><th></th></tr>
    <% produkter.forEach(produkt => { %>
      <tr>
        <td><%= produkt.produktnavn %></td>
        <td><%= produkt.pris %> kr</td>
        <td>
          <a href="/admin/<%= produkt.id %>/rediger">Rediger</a>
          <form method="POST" action="/admin/<%= produkt.id %>/slett"
style="display:inline">
            <button type="submit">Slett</button>
          </form>
        </td>
      </tr>
    <% }) %>
  </table>
</body>
</html>

```

Roast-tid 😊: se at "Slett" er en `<form method="POST">` og ikke en vanlig `<a>`-lenke? Det MÅ den være. Lenker er `GET`, og du skal **aldri slette data med en GET-forespørsel** — en nettleser-prefetch eller en crawler kan da slette hele databasen din helt av seg selv bare ved å "besøke" lenka. Endringer = `POST`. Alltid. 💀

Så `views/admin/skjema.ejs` — **ett skjema** som fungerer for både opprett OG rediger (smart gjenbruk):

```

<!DOCTYPE html>
<html lang="no">
<head><meta charset="UTF-8"><title>Produkt</title><link rel="stylesheet"
href="/style.css"></head>
<body>
  <a href="/admin">← Tilbake</a>
  <h1><%= produkt ? 'Rediger produkt' : 'Nytt produkt' %></h1>

  <form method="POST" action="<%= produkt ? '/admin/' + produkt.id +
'/rediger' : '/admin/ny' %>">
    <label>Navn
      <input name="produktnavn" value="<%= produkt ? produkt.produktnavn
: '' %>" required>
    </label>
    <label>Pris
      <input name="pris" type="number" step="0.01" value="<%= produkt ?
produkt.pris : '' %>" required>
    </label>
    <label>Beskrivelse
      <textarea name="beskrivelse"><%= produkt ? produkt.beskrivelse : ''
%></textarea>
    </label>
    <label>Bildefilnavn
      <input name="bilde" value="<%= produkt ? produkt.bilde : '' %>"
placeholder="roser.webp">
    </label>
    <button type="submit">Lagre</button>
  </form>
</body>
</html>

```

Trikset: er `produkt null` → tomt skjema som poster til `/admin/ny`. Er det fylt → forhåndsutfyllt skjema som poster til `/admin/:id/rediger`. Én fil, to jobber. 🎯

Nå: lås døra 🗝️

Admin fungerer nå — men hvem som helst på internett kan nå den. Vi lager en **middleware**: en liten "vakt" som kjører FØR admin-rutene og sjekker hvor forespørselen kommer fra. Legg denne funksjonen i `server.js`:

```
function kunLokaltNett(req, res, next) {
  const ip = req.ip; // f.eks. ::1, 127.0.0.1 eller 192.168.1.42

  const erLokal =
    ip === '::1' || // IPv6 localhost
    ip === '127.0.0.1' || // IPv4 localhost
    ip.startsWith('::ffff:127.') || // IPv4 mappet i IPv6
    ip.startsWith('192.168.') || // privat LAN
    ip.startsWith('::ffff:192.168.') ||
    ip.startsWith('10.') || // privat LAN
    ip.startsWith('::ffff:10.');
```

```
  if (erLokal) {
    next(); // alt ok → fortsett til admin-ruta
  } else {
    res.status(403).send('Tilgang nektet - admin er kun for bedriftens nett.');
```

```
  }
}
```

Og **aktiver den** ved å sette denne linja rett FØR alle admin-rutene:

```
app.use('/admin', kunLokaltNett);
```

Den kjører automatisk for alt som starter med `/admin`. `next()` betyr "alt good, gå videre til selve ruta". Dropper du `next()`, stopper forespørselen her. Det er hele middleware-konseptet: en mellomstasjon hver forespørsel må gjennom. 🚦

To viktige ting for deg spesielt:

Testing: Når du surfer fra din egen maskin er `req.ip` lik `::1`, så du slipper alltid inn under utvikling. For å *teste at sperra funker*, kan du midlertidig snu logikken (bytt `if (erLokal)` til `if (!erLokal)`) og se at du får **403**. Husk å snu tilbake. 😊

Når du drifter det på egen server (med din **nginx proxy manager**-erfaring 🙄): da ligger Express bak en reverse proxy, og `req.ip` viser da *proxys* IP, ikke kundens. Du må da sette

`app.set('trust proxy', true)` så Express leser den ekte IP-en fra `X-Forwarded-For`-headeren. Uten det blir ALT sett på som lokalt. Verdt å nevne i oppgaven.

Og den ærlige nyansen som gir deg ekstra poeng: middleware er **ett lag**. Den ordentlige måten å sikre admin på i produksjon er på **nettverksnivå** — firewall-regler eller å la admin kun lytte på det interne nettverksgrensesnittet, så trafikk utenfra aldri når serveren i det hele tatt. Middleware viser at du forstår prinsippet i kode; firewall er den som faktisk holder skurkene ute. Nevn begge, så viser du dybde. 🧠

Det var hele CRUD-en + nettverkssperra. 🔥 Når dette kjører har du faktisk dekket alle fire bulletpointsa i oppgaven. Vil du at vi rydder opp i CSS-en så det ser litt mindre "1998" ut, eller skal vi gå gjennom hva du bør skrive i selve **rapporten/dokumentasjonen**?